

Tokens, Contexto, Costo y Latencia

Lo que rompe (o salva) productos de IA

Ramsés Camas · Director del Bootcamp



Recap: Clase 1



Repo productivo

pyproject.toml, .env, estructura limpia



LLM Client

core/llm_client.py con Gemini configurado



Logging base

core/logger.py para trazabilidad



Primera llamada

CLI/endpoint que llama al LLM con logs

Objetivos de la Clase



- 1 Entender qué son los tokens y cómo impactan cada request
- 2 Comprender la ventana de contexto y sus limitaciones reales
- 3 Calcular el costo real de cada feature que usa LLMs
- 4 Medir y controlar la latencia (TTFT y throughput)
- 5 Construir tokenlab.py: tu herramienta de medición y budget

Tabla de Contenido

/01

Tokens

Qué son, cómo funcionan, ventana de contexto



/02

Costo

Pricing por token, budget por feature, escalamiento



/03

Latencia

TTFT, throughput, streaming, impacto en UX



/04

Control de Variación

Temperature, top-p, determinismo en producción



/01



Tokens

La unidad fundamental de los LLMs

¿Qué es un Token?



Un token es la unidad mínima de texto que un LLM puede procesar. No es una palabra, no es un carácter: es un fragmento que el modelo aprendió a reconocer durante su entrenamiento.

~4 caracteres

por token en inglés

~3 caracteres

por token en español

~0.75 palabras

por token (promedio)

Ejemplo de tokenización:

"Inteligencia artificial"

→ ["Int", "elig", "encia",
"art", "ificial"]

= 5 tokens

"AI is cool"

→ ["AI", " is", " cool"]

= 3 tokens



El idioma importa: español es más caro

Los tokenizadores están optimizados para inglés. El mismo mensaje en español genera más tokens = más costo y más latencia.

Texto	Idioma	Tokens	Ratio
"Hello, how are you?"	Inglés	6	1x
"Hola, ¿cómo estás?"	Español	9	1.5x
"Describe this image"	Inglés	4	1x
"Describe esta imagen"	Español	6	1.5x



Takeaway: Siempre mide tokens en TU idioma. Las estimaciones en inglés subestiman el costo real.

Ventana de Contexto



La ventana de contexto es el límite total de tokens que un modelo puede manejar en una sola llamada: input + output combinados.

Input Tokens (system + user + historial)

Output Tokens

¿Qué consume la ventana?

1. System prompt (instrucciones fijas)
2. Historial de conversación
3. Contexto RAG (documentos recuperados)
4. El mensaje del usuario
5. La respuesta generada

Más contexto =

Más costo

Más latencia

Menos espacio para output

Posible degradación



Ventana de Contexto por Modelo

Modelo	Context	Max Output	Equivalencia
Gemini 2.0 Flash	1M tokens	8,192	~5 novelas
Gemma 3 27B	128K tokens	8,192	~1 novela
GPT-4o	128K tokens	16,384	~1 novela
Claude Sonnet 4	200K tokens	64,000	~1.5 novelas

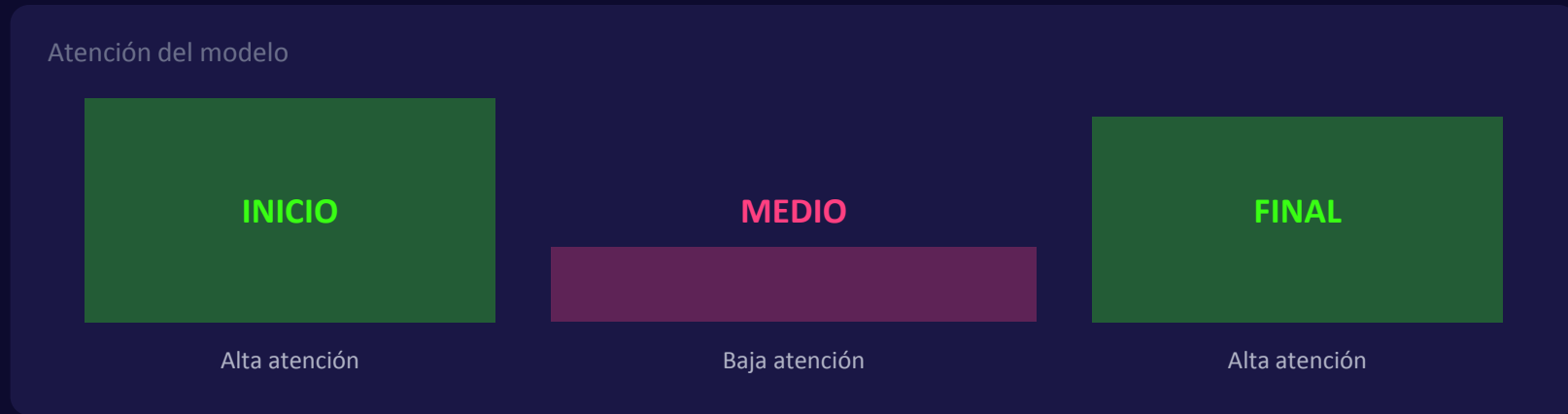


Context window grande $\neq$ úsalo todo. Enviar contexto innecesario incrementa costo, latencia y puede degradar la calidad ("lost in the middle").

"Lost in the Middle" Problem



Los LLMs prestan más atención al inicio y al final del contexto. La información en el medio tiende a ignorarse.



Tip: Pon la información más importante al inicio del prompt y la instrucción al final.

/02



Costo

El token es la moneda de los LLMs

Anatomía de un Request



INPUT TOKENS

(tú pagas por enviar)

System prompt

~200-500

Historial chat

~500-5,000

Contexto RAG

~1,000-10,000

Mensaje usuario

~20-200



OUTPUT TOKENS

(tú pagas por recibir — más caros)

La respuesta del modelo

Típicamente: ~100-4,000 tokens

**Output tokens cuestan 2-4x más
que input tokens**

Pricing: ¿Cuánto cuesta una llamada?



Modelo	Input/1M tok	Output/1M tok	Costo típico*
Gemini 2.0 Flash	FREE / \$0.10	FREE / \$0.40	\$0.00
GPT-4o	\$2.50	\$10.00	\$0.015
Claude Sonnet 4	\$3.00	\$15.00	\$0.021
GPT-4.1 mini	\$0.40	\$1.60	\$0.002

*Costo típico = 1 request con ~2K input + 500 output tokens



En este bootcamp usamos Gemini 2.0 Flash (free tier). 15 RPM, 1M tokens/min, 1,500 requests/día.
Suficiente para aprender sin gastar un centavo.

El Costo se Acumula Rápido



Ejemplo: Chatbot de atención al cliente con GPT-4o

1 conversación

~3K tokens → \$0.02

100 usuarios/día

~300K tokens → \$2/día

10,000 usuarios/día

~30M tokens → \$200/día

100,000 usuarios/día

~300M tokens → \$2,000/día

\$60,000 USD/mes

a 100K usuarios con GPT-4o

Budget por Feature



Cada feature de tu sistema tiene un budget de tokens. Si no lo defines, tu producto explota en costos.

	Clasificación de intent	In: ~200	Out: ~10	\$0.001	Alta
	Resumen de documento	In: ~4,000	Out: ~500	\$0.015	Media-Alta
	RAG: pregunta + contexto	In: ~3,000	Out: ~300	\$0.010	Alta
	Agente multi-step	In: ~10,000	Out: ~2,000	\$0.050	Variable

*Precios estimados con GPT-4o. Con Gemini free tier = \$0.

/03



Latencia

La diferencia entre un producto usable y uno abandonado

Métricas Clave de Latencia



TTFT

Time To First Token

Tiempo desde que envías el request hasta que recibes el primer token. Es lo que el usuario percibe como "velocidad de reacción".



TPS

Tokens Per Second (throughput)

Velocidad a la que el modelo genera tokens una vez que empezó. Determina cuánto tarda en completar la respuesta.

$$\text{Latencia total} = \text{TTFT} + (\text{output_tokens} / \text{TPS})$$

¿Qué Afecta la Latencia?



Tamaño del input	ALTO	Más tokens de entrada = más tiempo de prefill (TTFT aumenta)
Max output tokens	ALTO	Más tokens de salida = más tiempo total de generación
Modelo elegido	ALTO	Modelos más grandes = más lentos. GPT-4o vs Flash = 3-5x diferencia
Región del servidor	MEDIO	Latencia de red. US-East vs Asia puede agregar 200-500ms
Carga del proveedor	MEDIO	Rate limits y congestión impactan TTFT en horas pico
Streaming habilitado	BAJO	No cambia latencia total pero mejora percepción del usuario

Streaming vs No-Streaming



Sin Streaming

Request → espera... → respuesta completa

El usuario ve pantalla en blanco durante toda la generación.

Percepción: "está roto"

Útil: pipelines internos, batch, respuestas cortas

Con Streaming

Request → tokens llegan de a poco...

El usuario ve la respuesta construyéndose en tiempo real.

Percepción: "rápido e inteligente"

Útil: chatbots, UX de usuario final, respuestas largas



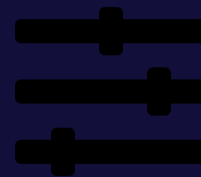
La latencia total es la misma. Streaming mejora la experiencia percibida, no la velocidad real.

Latencia en Producción: Números Reales

Escenario	TTFT	Total	UX
Clasificación simple	~200ms	~400ms	Instantáneo
Chat con RAG	~800ms	~3-5s	Aceptable con streaming
Agente multi-tool	~1-2s	~10-30s	Necesita progress bar
Generación de reporte	~2-5s	~30-120s	Async obligatorio

Regla de UX: <200ms instantáneo | <1s fluido | <5s aceptable | >10s frustración

/04



Control de Variación

Determinismo y reproducibilidad en producción

Temperature: Creatividad vs Precisión



La temperature controla qué tan creativa o determinista es la salida del modelo. Rango: 0.0 a 2.0.



temp = 0.0

Determinista

Clasificación, extracción de datos, JSON parsing, funciones

Producción: tareas con respuesta correcta

temp = 0.7

Balanceado

Chat conversacional, resúmenes, explicaciones, redacción

Default para la mayoría de aplicaciones

temp = 1.2+

Creativo

Brainstorming, escritura creativa, exploración de ideas

Cuidado: puede alucinar más

Top-P y Top-K: Filtros de Vocabulario

top_p (Nucleus Sampling)

Considera solo tokens cuya probabilidad acumulada suma p.

top_p = 0.1 → solo los más probables

top_p = 0.9 → vocabulario más amplio

Default: 1.0 (sin filtro)

top_k

Considera solo los K tokens más probables para el siguiente paso.

top_k = 1 → greedy (siempre el más probable)

top_k = 40 → vocabulario moderado

No todos los proveedores lo exponen

En producción:

Ajusta **temperature** O **top_p**, no ambos. Para extracción/clasificación: **temp=0** y listo.



Determinismo en Producción

¿Puedes obtener la misma respuesta dos veces? Spoiler: no siempre, pero puedes acercarte.

config para máximo determinismo

```
generation_config = {  
    "temperature": 0.0,  
    "top_p": 1.0,  
    "top_k": 1,  
    "max_output_tokens": 256,  
}
```

Aun con temp=0:

1. Actualizaciones del modelo pueden cambiar outputs
2. Batching del servidor puede introducir variaciones
3. Diferentes contextos = diferentes resultados
4. Siempre valida con tests, nunca asumas determinismo



max_output_tokens: Tu Mejor Amigo

Limitar los tokens de salida es la forma más directa de controlar costo y latencia por request.

Clasificación (sí/no)	10	~100ms	Mínimo
Extracción JSON	256	~500ms	Bajo
Resumen corto	512	~1s	Bajo
Respuesta de chat	1,024	~2-3s	Medio
Reporte largo	4,096	~8-15s	Alto



Nunca dejes max_output_tokens sin definir en producción. Un LLM sin límite puede generar miles de tokens innecesarios.

Todo Está Conectado



"Si no puedes medir el costo de cada request, no estás haciendo ingeniería de IA. Estás rezando."



Principio del AI Engineer

Hora de Código: tokenlab.py



Vamos a construir la herramienta que usaremos el resto del bootcamp para medir y controlar cada request.



`count_tokens(text)`

Contar tokens de cualquier texto usando la API de Gemini



`estimate_cost(input, output)`

Calcular costo estimado por modelo y por request



`measure_latency(request)`

Medir TTFT y TPS de cada llamada

Ubicación: `core/tokenlab.py` → Reuso en clases 3, 5, 14-16

Demo en Vivo



core/tokenlab.py (fragmento)

```
import time
from core.llm_client import get_model

def measure_request(prompt: str):
    model = get_model()
    t0 = time.perf_counter()
    response = model.generate_content(prompt)
    t1 = time.perf_counter()

    input_tok = response.usage_metadata
                    .prompt_token_count
    output_tok = response.usage_metadata
                    .candidates_token_count

    return {
        "latency": t1 - t0,
        "input_tokens": input_tok,
        "output_tokens": output_tok,
    }
```

Ejercicio Práctico



Tienes 25 minutos. Completa `tokenlab.py` y experimenta.

1

Contar tokens

5 min

Implementa `count_tokens()` usando `model.count_tokens()` de Gemini.

2

Comparar idiomas

5 min

Mide tokens de la misma oración en español e inglés. ¿Cuál gasta más?

3

Medir latencia

10 min

Envía 3 prompts de diferente tamaño. Registra latencia total. ¿Hay correlación?

4

Budget check (bonus)

5 min

Crea función que valide si un request excede el budget de tokens del feature.

Resumen de Hoy



Tokens son la moneda: input + output = costo + latencia

Ventana de contexto: úsala inteligentemente, no la llenes por llenar

El costo escala linealmente con usuarios. Sin budget = desastre

TTFT importa más que latencia total (percepción del usuario)

max_output_tokens + temperature = tus controles principales



Próxima clase: **Clase 3 — Prompt Engineering para Sistemas**

Templates, few-shot, chaining y prompts evaluables (no vibes).

Bootcamp AI Engineer

Clase 2 Completada

Nos vemos en la Clase 3



Ramsés Camas · Director del Bootcamp · Código Facilito